

AML Mid-Sem · One-Shot Notes

Advanced Machine Learning – all 15 syllabus topics, revised in one sitting

How to read these notes

Go top to bottom once. Every topic has the same order: **big idea** → **key concepts** → **formulas** → **derivation (every step)** → **a fully worked calculation** → **exam traps**.

yellow = must remember

green = the "why"

pink = common mistake

Red boxes = formulas. Blue boxes = derivations: each numbered line follows from the one above, and the grey 📝 note says what was done.

➤ Contents

- | | |
|--|--|
| 1. ML Project Lifecycle (Part 1) | 9. Bias, Variance & Tradeoff |
| 2. ML Project Lifecycle (Part 2) | 10. Feature Selection |
| 3. Simple Linear Regression (OLS) | 11. Principal Component Analysis |
| 4. Multiple Linear Regression (OLS) | 12. Regularization |
| 5. Batch Gradient Descent | 13. Time Series Analysis |
| 6. Stochastic & Mini-Batch GD | 14. MLE & Logistic Regression |
| 7. Evaluation Metrics | 15. GD on Logistic Regression & Multiclass |
| 8. Polynomial Regression & Assumptions | 16. Last page: formulas at a glance |

Maths you need (nothing more)

- **Sum:** $\sum_{i=1}^n x_i = x_1 + x_2 + \dots + x_n$. **Mean:** $\bar{x} = \frac{1}{n} \sum x_i$, so $\sum x_i = n\bar{x}$.
- **Derivative rules:** $\frac{d}{dx} x^2 = 2x$; $\frac{d}{dx} (a - bx)^2 = 2(a - bx) \cdot (-b)$ (chain rule: outside $\times$ inside); $\frac{d}{dx} \ln x = \frac{1}{x}$; $\frac{d}{dx} e^x = e^x$.
- **Minimum of a smooth bowl:** set the derivative to 0 and solve.
- **Matrix sizes:** $(a \times b)(b \times c) = (a \times c)$. Transpose flips: $(AB)^T = B^T A^T$.
- **2x2 inverse:** $\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$ (swap the diagonal, negate the off-diagonal, divide by the determinant).
- **Logs:** $\ln(ab) = \ln a + \ln b$, $\ln a^k = k \ln a$, $\ln 1 = 0$, $\ln 2 = 0.693$, $\ln 0.1 = -2.303$.

1 ML Project Lifecycle – Part 1

Big idea: an ML project is a pipeline of phases. Phases 1–4: Problem definition → Data collection → EDA → Preprocessing. (Part 2 covers phases 5–11.)

➤ Phase 1 – Problem definition (5-step blueprint)

Step	What it means
1. Understand the problem	Find the exact pain point (e.g. premium users cancelling = churn).
2. Set quantifiable goals	A measurable target: "reduce churn by 10% in 2 quarters", not "make it better".
3. Assess ML feasibility	Is ML even needed? Is there data? "Don't use ML" is a valid answer (if a SQL rule solves it).
4. Identify constraints	Budget, latency (<10 ms?), privacy laws (GDPR) — legal limits are constraints .
5. Stakeholder alignment	Everyone agrees what "success" means.

Rigorous problem statement

"Using [data], predict [target] within [time window], achieving [metric ≥ k]."

e.g. "Using clickstream + purchase data, predict which users stop shopping within 3 months."

➤ Phase 2 – Data collection

Sources: SQL warehouses, public repositories (Kaggle, UCI), APIs, web scraping, manual labelling. **Garbage In = Garbage Out**: the model can't be better than its data.

Structure	Example	Feature type	Example
Structured	SQL table (rows × columns)	Nominal (no order)	city, colour, blood group
Unstructured	images, audio, free text	Ordinal (order)	HS < Bachelor < Master < PhD
Semi-structured	JSON, XML (keys/tags, no fixed schema)	Numerical discrete	no. of rooms
Time-series	indexed by time (daily prices)	Numerical continuous	income, temperature

A number used as an ID (roll no., PIN, phone) is nominal, not numerical.

➤ Phase 3 – Exploratory Data Analysis (EDA)

- **Univariate** (1 variable): distribution, mean, median, outliers → histogram, box plot.
- **Bivariate** (2 variables): relationship → scatter plot, correlation.
- **Multivariate** (many): interactions → correlation heatmap, pair plot.
- Histogram = one **numerical** variable; bar chart = **categorical** counts. Box plot shows the **median** (not mean).
- **Class imbalance**: 95% "no fraud" → a model that always says "no" gets 95% accuracy but catches 0 **fraud**. Majority accuracy = n_{majority}/n .

Quartiles, IQR, outlier fences

$$\text{IQR} = Q_3 - Q_1 \quad \text{outlier if } x < Q_1 - 1.5 \text{ IQR} \text{ or } x > Q_3 + 1.5 \text{ IQR}$$

Q_1 = median of the lower half, Q_3 = median of the upper half (median itself excluded when n is odd).

Worked: fences

Data: 10, 20, 30, 40, 100 (sorted, $n = 5$). Median = 30 (middle value).

- 1 Lower half = 10, 20 $\rightarrow Q_1 = (10 + 20)/2 = 15$. Upper half = 40, 100 $\rightarrow Q_3 = (40 + 100)/2 = 70$.
- 2 $\text{IQR} = 70 - 15 = 55$; $1.5 \times 55 = 82.5$.
- 3 Lower fence = $15 - 82.5 = -67.5$; upper fence = $70 + 82.5 = 152.5$. Nothing lies outside, so **no outlier** by this rule (100 is large but inside the fence).

➤ Phase 4 – Preprocessing

(a) Missing values

Method	Use when
Delete rows	very few rows missing
Delete column	column mostly empty ($> \sim 80\%$)
Mean imputation	numerical, no outliers
Median imputation	numerical with outliers / skew (median is robust)
Mode imputation	categorical (mean of "IT" and "HR" is meaningless)

Mean of a column with gaps uses only the **observed** values: $[2, ?, 4, ?, 9] \rightarrow \text{mean} = 15/3 = 5 \rightarrow [2, 5, 4, 5, 9]$.

(b) Inconsistent data

Duplicates (drop), inconsistent spellings/case ("delhi", "Delhi"), mixed units (kg vs g) $\rightarrow$ standardise.

(c) Feature scaling – 4 scalars

Scaling formulas

$$\text{Min-Max: } x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \in [0, 1]$$

$$\text{Max-Abs: } x' = \frac{x}{\max |x|} \in [-1, 1]$$

$$\text{Z-score: } z = \frac{x - \mu}{\sigma}$$

$$\text{Robust: } x' = \frac{x - \text{median}}{\text{IQR}}$$

σ here is the **population** std: $\sigma = \sqrt{\frac{1}{n} \sum (x_i - \mu)^2}$ (divide by n). StandardScaler also divides by n .

Worked: scale $x = 30$ in the column 10, 20, 30, 40, 100

- 1 **Min-Max:** $(30 - 10)/(100 - 10) = 20/90 = 0.222$.
- 2 **Z-score:** $\mu = (10 + 20 + 30 + 40 + 100)/5 = 200/5 = 40$. Deviations: $-30, -20, -10, 0, 60$. Squares: $900, 400, 100, 0, 3600 \rightarrow \text{sum } 5000$. $\sigma^2 = 5000/5 = 1000$, $\sigma = \sqrt{1000} = 31.62$. $z = (30 - 40)/31.62 = -0.316$.
- 3 **Max-Abs:** $30/100 = 0.3$.
- 4 **Robust:** median 30, IQR 55 (from above): $(30 - 30)/55 = 0$.

Why Robust exists: the outlier 100 squashes Min-Max (all normal values crowd near 0); median and IQR ignore the extreme value.

Who needs scaling? Gradient descent, KNN, SVM, PCA, regularised models. **Trees don't.** Fit the scaler on **train only**, then transform test with the same numbers.

(d) Categorical encoding

Encoding	How	Use for	Columns
Label	each category $\rightarrow$ integer (sklearn: alphabetical)	target labels / trees	1
Ordinal	integers in an order you give	ordered categories	1
One-hot	one 0/1 column per category	nominal, few categories	N
Binary	id $\rightarrow$ binary digits, one column per bit	nominal, many categories	$\lceil \log_2 N \rceil$

Worked: binary encoding of 5 animals

$N = 5$, ids 0-4 $\rightarrow \lceil \log_2 5 \rceil = \lceil 2.32 \rceil = 3$ columns. Cat = 0 = 000, Dog = 1 = 001, Cow = 2 = 010, Hen = 3 = 011, Bird = 4 = 100. (One-hot would need 5.) If ids start at 1, Bird = 5 = 101 still needs 3 bits, but $N = 8$ would need 4.

⚠ Exam traps

- JSON / XML = **semi-structured**.
- Label-encoding a nominal feature for linear models/KNN invents a fake order ("Red < Green").
- LabelEncoder on ['low','high','medium'] gives high = 0, low = 1, medium = 2 (alphabetical).
- EDA comes **before** preprocessing.

2 ML Project Lifecycle – Part 2

Phase	Name	One line
5	Data splitting	train / validation / test — the "golden barrier"
6	Feature engineering	create better inputs
7	Baseline	the simplest model you must beat
8	Modelling loop	select, tune, train
9	Offline evaluation	score once on the test set; map to business value
10	Deployment	ship the model
11	Post-deployment	monitor, detect drift, retrain

➤ Parameters vs hyperparameters

Key

Parameters are learned **during** training (weights w , intercept w_0). **Hyperparameters** are set **before** training by you (learning rate α , max_depth, n_estimators, λ , degree, batch size). **Hyperparameters are tuned on the validation set**, never on the test set.

➤ Phase 5 – Splitting and data leakage

- Typical split **70 / 15 / 15**. Train = learn parameters; validation = choose hyperparameters/model; test = **one final, untouched check**.
- **Data leakage** = information from validation/test (or from the future) sneaks into training → scores look great, real performance is worse.
- **Leak examples**: scaling or imputing with the mean of the whole dataset before splitting; tuning on the test set; a feature computed from the target; using future values in time series.
- **Golden rule**: split first, then fit every preprocessing step (imputer, scaler, encoder, feature selection) on train only.

➤ Phase 6 & 7 – Feature engineering and baseline

Feature creation: interactions $x_1 \times x_2$, ratios, date parts (day of week), polynomial terms. Baseline: **regression** → always predict $\bar{y}_{\text{train}}$; **classification** → always predict the majority class. A new model is useful only if it clearly beats the baseline.

➤ Phase 8 – The modelling loop

initialise weights → predict (forward) → compute loss → compute gradient → update weights → check convergence → repeat

Loss + update (preview of L5)

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad w \leftarrow w - \alpha \frac{\partial \text{Loss}}{\partial w}$$

Converged = the loss stops changing (not "loss = 0"). Learning rate too big → overshoots/diverges; too small → painfully slow.

➤ Phase 9 – Offline evaluation and business mapping

Evaluate the locked model on the test set (MAE, RMSE, R^2 for regression). Then translate into money:

Worked: is the better model worth it?

Baseline MAE = 12, new MAE = 11 (1 unit better). 1 unit of error costs ₹80,000 per farm; 500 farms; training costs ₹15 lakh.

- ① Saving = improvement × cost per unit × volume = $1 \times 80,000 \times 500 = ₹4,00,00,000$ (4 crore).
- ② 4 crore $\gg$ 15 lakh → **deploy**. Always convert the metric gain into money before deciding.

➤ Phase 10 – Deployment

Save the model (.pkl pickle) → serve via an **API** (Flask/FastAPI), the **cloud** (SageMaker, Azure), or the **edge** (phone/IoT). **Batch** predictions = periodic, many rows (nightly recommendations); **real-time** = instant per request (fraud at checkout). Chosen by latency need.

➤ Phase 11 – Post-deployment

- **Model drift**: accuracy falls because the world changed (e.g. 2020 pandemic vs a 2010–2019 travel model). The code is fine; the data changed.
- **Monitoring** (Prometheus, MLflow) watches live data and metrics.
- **Periodic retraining** = on a schedule, regardless of performance. **Triggered retraining** = when a metric drops below a threshold τ .
- "Deploy and walk away" is always wrong for ML. The macro loop goes back to the start.

⚠ Exam traps

- Learning rate is a **hyperparameter**, even though it controls training.
- Imputing/scaling before splitting = leakage, even if the model never "sees" test rows.
- R^2 can be negative on the test set (worse than predicting the mean).

3 Simple Linear Regression (OLS)

Big idea: fit the straight line $\hat{y} = mx + c$ that makes the **sum of squared vertical errors as small as possible**. OLS gives the answer in one formula (closed form) — no iteration.

► Residuals and choosing the loss

Residual

$$e_i = y_i - \hat{y}_i = y_i - (mx_i + c) \quad (\text{actual} - \text{predicted, a vertical gap})$$

Loss	Problem / property
$\sum e_i$ (signed)	+ and - errors cancel ; total 0 doesn't mean a good fit
$\sum e_i $ (absolute)	robust to outliers, but has a kink at 0 (not differentiable there) → no clean formula
$\sum e_i^2$ (squared) ✓	smooth everywhere, so set derivatives to 0 → closed form. But it amplifies outliers (error 10 counts 100x)

OLS loss (error surface)

$$E(m, c) = \sum_{i=1}^n (y_i - mx_i - c)^2$$

It is a convex **bowl** over the (m, c) plane → exactly one lowest point, found where $\frac{\partial E}{\partial m} = 0$ and $\frac{\partial E}{\partial c} = 0$.

Derivation 1 – the intercept c

1 $E = \sum (y_i - mx_i - c)^2$

☞ start from the loss

2 $\frac{\partial E}{\partial c} = \sum 2(y_i - mx_i - c) \cdot (-1)$

☞ chain rule: derivative of $(\text{stuff})^2$ is $2 \cdot \text{stuff} \cdot (\text{derivative of stuff})$; the derivative of $(y_i - mx_i - c)$ w.r.t. c is -1

3 $-2 \sum (y_i - mx_i - c) = 0$

☞ set to zero for the minimum

4 $\sum y_i - m \sum x_i - \sum c = 0$

☞ divide by -2 and split the sum into three sums

5 $\sum y_i - m \sum x_i - nc = 0$

☞ c is the same for all n rows, so $\sum c = nc$ (not $c!$)

6 $c = \frac{\sum y_i}{n} - m \frac{\sum x_i}{n}$

☞ move nc to the right, divide by n

7 $c = \bar{y} - m\bar{x}$

☞ $\frac{\sum y_i}{n} = \bar{y}$, $\frac{\sum x_i}{n} = \bar{x}$

Meaning: put $x = \bar{x}$ in the line: $\hat{y} = m\bar{x} + \bar{y} - m\bar{x} = \bar{y}$. **The OLS line always passes through $(\bar{x}, \bar{y})$.**

Also step 3 says $\sum e_i = 0$: residuals sum to zero.

Derivation 2 – the slope m

- ① $y_i - mx_i - c = y_i - mx_i - \bar{y} + m\bar{x}$
 $\hookrightarrow$ substitute $c = \bar{y} - m\bar{x}$ (this removes c , leaving one unknown)
- ② $= (y_i - \bar{y}) - m(x_i - \bar{x})$
 $\hookrightarrow$ group the y -terms and the m -terms
- ③ Let $a_i = x_i - \bar{x}$ and $b_i = y_i - \bar{y}$. Then $E(m) = \sum (b_i - ma_i)^2$.
 $\hookrightarrow$ short names for the deviations from the mean
- ④ $\frac{dE}{dm} = \sum 2(b_i - ma_i) \cdot (-a_i)$
 $\hookrightarrow$ chain rule again; the derivative of $(b_i - ma_i)$ w.r.t. m is $-a_i$
- ⑤ $-2 \sum (a_i b_i - ma_i^2) = 0$
 $\hookrightarrow$ multiply out $(b_i - ma_i)(-a_i)$ and set to 0
- ⑥ $\sum a_i b_i = m \sum a_i^2$
 $\hookrightarrow$ divide by -2 , move the m -term to the other side
- ⑦ $m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}} = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$

Minimum check: $\frac{d^2 E}{dm^2} = 2 \sum a_i^2 > 0 \rightarrow$ bowl opens upward $\rightarrow$ minimum.

OLS recipe

① $\bar{x}, \bar{y} \rightarrow$ ② $a_i = x_i - \bar{x}, b_i = y_i - \bar{y} \rightarrow$ ③ $S_{xy} = \sum a_i b_i, S_{xx} = \sum a_i^2 \rightarrow$ ④ $m = S_{xy}/S_{xx} \rightarrow$ ⑤ $c = \bar{y} - m\bar{x}$.

Shortcut form: $m = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$ (same answer, no means needed).

Worked: $x = 1, 2, 3, 4, 5$ and $y = 2, 4, 5, 4, 5$ (the course lab)

$\bar{x} = 15/5 = 3, \bar{y} = 20/5 = 4$.

x	y	$a = x - 3$	$b = y - 4$	$a \cdot b$	a^2
1	2	-2	-2	4	4
2	4	-1	0	0	1
3	5	0	1	0	0
4	4	1	0	0	1
5	5	2	1	2	4
sums (check: $\Sigma a = 0, \Sigma b = 0$)				6	10

$m = 6/10 = 0.6, c = 4 - 0.6 \times 3 = 2.2 \rightarrow \hat{y} = 0.6x + 2.2$. Prediction at $x = 6$: $0.6 \times 6 + 2.2 = 5.8$.

⚠ Exam traps

- Compute **m first**, then c . The denominator is $\sum (x - \bar{x})^2$ — x only (it's NOT the covariance).
- All x equal $\rightarrow S_{xx} = 0 \rightarrow$ slope **undefined** (the lab returns $[-1.0, -1.0]$).
- Why substitute c first? To reduce 2 unknowns to 1, not because " c must come first in ML".
- One outlier can swing the slope a lot (squared loss). Absolute loss is the robust one.
- Two inputs $\rightarrow$ a **plane** $\hat{y} = m_1 x_1 + m_2 x_2 + c$ (3 parameters). Parameters = features + 1.

4 Multiple Linear Regression (OLS)

Big idea: same as SLR but with p features. Write everything with matrices and get the Normal Equation $\beta = (X^T X)^{-1} X^T y$.

Model in matrix form

$$\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip} \iff \hat{\mathbf{y}} = X\beta$$

$$X = \begin{bmatrix} 1 & x_{11} & \dots & x_{1p} \\ 1 & x_{21} & \dots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & \dots & x_{np} \end{bmatrix}_{n \times (p+1)} \quad \beta = \begin{bmatrix} \beta_0 \\ \vdots \\ \beta_p \end{bmatrix}_{(p+1) \times 1}$$

The **column of 1s** carries the intercept. Sizes: $X^T X$ is $(p+1) \times (p+1)$, $X^T y$ is $(p+1) \times 1$ — they do not depend on n . p features $\rightarrow p+1$ parameters.

Derivation – the Normal Equation

- 1 $\mathbf{e} = \mathbf{y} - X\beta$
 $\hookrightarrow$ residual vector ($n \times 1$)
- 2 $L = \mathbf{e}^T \mathbf{e} = \sum e_i^2 = (\mathbf{y} - X\beta)^T (\mathbf{y} - X\beta)$
 $\hookrightarrow e^T e = \text{sum of squares (a single number)}$. Note ee^T would be an $n \times n$ matrix — wrong
- 3 $= (\mathbf{y}^T - \beta^T X^T) (\mathbf{y} - X\beta)$
 $\hookrightarrow$ transpose of a difference = difference of transposes; $(X\beta)^T = \beta^T X^T$ (order flips)
- 4 $= \mathbf{y}^T \mathbf{y} - \mathbf{y}^T X\beta - \beta^T X^T \mathbf{y} + \beta^T X^T X\beta$
 $\hookrightarrow$ multiply out like $(a-b)(c-d)$
- 5 $= \mathbf{y}^T \mathbf{y} - 2\mathbf{y}^T X\beta + \beta^T X^T X\beta$
 $\hookrightarrow$ the two middle terms are the same 1×1 number (a number equals its own transpose), so they add up to -2 of it
- 6 Use the 3 identities: $\frac{\partial}{\partial \beta}(\text{const}) = 0$, $\frac{\partial}{\partial \beta}(\mathbf{a}^T \beta) = \mathbf{a}$, $\frac{\partial}{\partial \beta}(\beta^T A \beta) = 2A\beta$ (A symmetric)
 $\hookrightarrow \mathbf{y}^T \mathbf{y}$ has no β ; $\mathbf{y}^T X\beta = (X^T \mathbf{y})^T \beta$ so $\mathbf{a} = X^T \mathbf{y}$; $A = X^T X$ is symmetric
- 7 $\nabla L = 0 - 2X^T \mathbf{y} + 2X^T X\beta$
 $\hookrightarrow$ differentiate term by term (keep the -2)
- 8 $2X^T X\beta = 2X^T \mathbf{y} \Rightarrow \boxed{X^T X\beta = X^T \mathbf{y}}$
 $\hookrightarrow$ set the gradient to 0 and divide by 2 — this is the Normal Equation
- 9 $\boxed{\beta^* = (X^T X)^{-1} X^T \mathbf{y}}$
 $\hookrightarrow$ multiply both sides on the left by $(X^T X)^{-1}$ (if it exists)

Why the identities are true (2 parameters, so you can see it)

Identity 2: $\mathbf{a}^T \beta = a_1 \beta_1 + a_2 \beta_2$. Then $\partial / \partial \beta_1 = a_1$ and $\partial / \partial \beta_2 = a_2 \rightarrow$ the gradient is $(a_1, a_2) = \mathbf{a}$.

Identity 3: with $A = \begin{bmatrix} p & q \\ q & r \end{bmatrix}$: $\beta^T A \beta = p\beta_1^2 + 2q\beta_1\beta_2 + r\beta_2^2$. Then $\partial / \partial \beta_1 = 2p\beta_1 + 2q\beta_2$ and

$\partial / \partial \beta_2 = 2q\beta_1 + 2r\beta_2 \rightarrow$ that's $2 \begin{bmatrix} p & q \\ q & r \end{bmatrix} \begin{bmatrix} \beta_1 \\ \beta_2 \end{bmatrix} = 2A\beta$. This needed A symmetric (both off-diagonals = q). $X^T X$ always is, because $(X^T X)^T = X^T X$.

Worked: fit $y = \beta_0 + \beta_1 x$ to $(0, 1), (1, 2), (2, 6)$

① $X = \begin{bmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}, y = \begin{bmatrix} 1 \\ 2 \\ 6 \end{bmatrix}.$

② $X^T X = \begin{bmatrix} 1+1+1 & 0+1+2 \\ 0+1+2 & 0+1+4 \end{bmatrix} = \begin{bmatrix} 3 & 3 \\ 3 & 5 \end{bmatrix}$
 ☞ top-left = n , off-diagonal = Σx , bottom-right = Σx^2

③ $X^T y = \begin{bmatrix} 1+2+6 \\ 0 \cdot 1 + 1 \cdot 2 + 2 \cdot 6 \end{bmatrix} = \begin{bmatrix} 9 \\ 14 \end{bmatrix}$
 ☞ top = Σy , bottom = Σxy

④ $\det = 3 \cdot 5 - 3 \cdot 3 = 6; (X^T X)^{-1} = \frac{1}{6} \begin{bmatrix} 5 & -3 \\ -3 & 3 \end{bmatrix}$
 ☞ 2x2 rule: swap the diagonal, negate the off-diagonal, divide by det

⑤ $\beta = \frac{1}{6} \begin{bmatrix} 5 \cdot 9 - 3 \cdot 14 \\ -3 \cdot 9 + 3 \cdot 14 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 3 \\ 15 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 2.5 \end{bmatrix} \rightarrow \hat{y} = 0.5 + 2.5x$

► When the Normal Equation fails

- $(X^T X)^{-1}$ exists only if the columns of X are linearly independent ($\det X^T X \neq 0$). If a column is an exact combination of others ($x_2 = 2x_1$, or °C and °F together, or a duplicated 1s column) $\rightarrow$ **singular**, no inverse. This is **perfect multicollinearity**.
- **Cost:** inverting a $k \times k$ matrix ($k = p + 1$) by Gauss-Jordan takes 3 nested loops: pivot columns (k) $\times$ rows to eliminate (k) $\times$ entries in the augmented row ($\sim 2k$) $\approx 2k^3 = O(k^3)$. $p = 100 \rightarrow$ about 10^6 operations; doubling $p \rightarrow 8\times$ the cost. That is why gradient descent is used for huge p .
- Too few rows ($n < p + 1$) also makes $X^T X$ singular.

⚠ Exam traps

- Order matters: $(X^T X)^{-1} X^T y$, not $X^T (X^T X)^{-1} y$.
- MLR is "linear" because it is linear in the **parameters** β .
- NumPy: predictions = $X @ \text{theta}$. sklearn adds the intercept itself (don't add a 1s column there too).
- Adding a feature never increases training error (it can only stay the same or drop).

5 Batch Gradient Descent

Big idea: instead of solving the normal equation, start with any θ and repeatedly step downhill on the loss surface. "Batch" = every step uses **all** training rows.

➤ Why not just use OLS?

1. **Cost:** inverting $X^T X$ is $O(p^3)$ — impossible for millions of features.
2. **Invertibility:** collinear columns $\rightarrow X^T X$ singular $\rightarrow$ no inverse. (GD still runs, but the solution is not unique.)

➤ The update rule

Gradient descent

$$\theta_{\text{new}} = \theta_{\text{old}} - \alpha \nabla L(\theta_{\text{old}}) \quad \text{i.e.} \quad \theta_j \leftarrow \theta_j - \alpha \frac{\partial L}{\partial \theta_j}$$

- The **gradient** ∇L points uphill (direction of steepest increase). **Minus** $\rightarrow$ we go downhill. The steeper the slope, the bigger the step.
- **α (learning rate)** = step size, a hyperparameter. Too small $\rightarrow$ slow; too large $\rightarrow$ overshoots and **diverges** (loss grows).
- **Simultaneous update:** compute **all** partial derivatives using the **old** θ , then update all θ_j together. Updating θ_0 first and using the new θ_0 for θ_1 is wrong.
- **Initialisation:** zeros are fine for linear/logistic regression (convex bowl, one minimum).
- **Epoch** = one full pass over the data. In BGD, **1 epoch = 1 update**.

Derivation — gradient of the MSE

m rows, prediction $\hat{y}_i = \theta^T \mathbf{x}_i = \theta_0 + \theta_1 x_{i1} + \dots$ (with $x_{i0} = 1$).

① $L(\theta) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2$

☞ MSE (squaring makes $\hat{y} - y$ and $y - \hat{y}$ the same)

② $\frac{\partial L}{\partial \theta_j} = \frac{1}{m} \sum_i 2(\hat{y}_i - y_i) \cdot \frac{\partial \hat{y}_i}{\partial \theta_j}$

☞ chain rule on each squared term

③ $\frac{\partial \hat{y}_i}{\partial \theta_j} = x_{ij}$

☞ in $\theta_0 + \theta_1 x_{i1} + \dots$ only the θ_j term contains θ_j , and its coefficient is x_{ij}

④ $\frac{\partial L}{\partial \theta_j} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij}$

☞ for θ_0 the "x" is 1, so it is just $\frac{2}{m} \sum (\hat{y}_i - y_i)$

⑤ Stack all j into a vector: $\nabla L = \frac{2}{m} X^T (X\theta - \mathbf{y})$

☞ row j of X^T times the residual vector = $\sum_i x_{ij}(\hat{y}_i - y_i)$

Some books use $L = \frac{1}{2m} \sum (\cdot)^2$; then the 2 cancels and $\nabla L = \frac{1}{m} X^T (X\theta - \mathbf{y})$. Same minimum, only the step size changes.

Worked: one BGD epoch on (1,2), (2,4), (3,6), (4,8), $\theta = (0, 0)$, $\alpha = 0.1$

- 1 Predictions with $\theta = (0,0)$: all 0. Errors $\hat{y} - y = -2, -4, -6, -8$.
- 2 **Course-quiz convention** (average of e and $e \cdot x$, no 2): $g_0 = \frac{-2-4-6-8}{4} = -5$; $g_1 = \frac{(-2)(1)+(-4)(2)+(-6)(3)+(-8)(4)}{4} = \frac{-60}{4} = -15$.
- 3 Update: $\theta_0 = 0 - 0.1(-5) = 0.5$, $\theta_1 = 0 - 0.1(-15) = 1.5 \rightarrow \theta = (0.5, 1.5)$ after **one** update.
- 4 With the worksheet's full MSE (factor $\frac{2}{m}$) the gradient doubles to $(-10, -30)$ and $\theta = (1, 3)$.
Check which convention the question/options use.

Worked: lab model $y = w \cdot x$ (no bias)

$x = [1, 2, 3]$, $y = [2, 4, 6]$, $w = 0$, $lr = 0.01$, $dw = \text{mean}(2(wx_i - y_i)x_i)$.

Terms: $2(0 - 2)(1) = -4$, $2(0 - 4)(2) = -16$, $2(0 - 6)(3) = -36 \rightarrow \text{mean} = -56/3 = -18.667 \rightarrow w = 0 - 0.01(-18.667) = 0.1867$. Repeat for each epoch; it converges to $w = 2$.

➤ Learning rate and convergence

Why a big α diverges (1-D bowl $L = a\theta^2$)

- 1 $\frac{dL}{d\theta} = 2a\theta \rightarrow \theta_{\text{new}} = \theta - \alpha \cdot 2a\theta = (1 - 2a\alpha)\theta$
- 2 Each step multiplies θ by $(1 - 2a\alpha)$. It shrinks toward 0 (the minimum) only if $|1 - 2a\alpha| < 1$, i.e. $0 < \alpha < 1/a$.
- 3 $\alpha > 1/a \rightarrow |1 - 2a\alpha| > 1 \rightarrow \theta$ jumps across the bowl further each time $\rightarrow$ **diverges**. Between $\frac{1}{2a}$ and $\frac{1}{a}$ it zig-zags but still converges.

- **Choose α on the validation loss** (never the test set). Typical grid: 0.001, 0.01, 0.1, 1.
- **Feature scaling**: features on very different scales (attendance 70-100 vs hours 1-5) make a stretched bowl; the large feature gets huge gradients, so α must be tiny and progress is slow (zig-zag). Standardise first.
- Convex MSE + suitable $\alpha \rightarrow$ BGD reaches the **same global minimum as OLS**.

BGD pros / cons

+ exact gradient, smooth stable path, deterministic. - one update needs a full pass: **high cost per update**, memory & I/O pressure (data bigger than RAM), slow feedback (can't adapt quickly, e.g. fraud), **redundant work** (near-identical rows).

⚠ Exam traps

- Subtract $\alpha \times$ gradient (adding goes uphill). If you write the error as $y - \hat{y}$, the sign flips.
- BGD does not invert $X^T X$.
- In L4 "m" = number of features; in LS "m" = number of rows. Read carefully.

6 Stochastic & Mini-Batch Gradient Descent

Big idea: BGD waits for all m rows before one step. SGD steps after every single row; mini-batch steps after every b rows.

The three update rules

$$\text{BGD: } \theta \leftarrow \theta - \alpha \frac{1}{m} \sum_{i=1}^m \nabla J_i \quad \text{SGD: } \theta \leftarrow \theta - \alpha \nabla J_{i_t} \quad \text{Mini-batch: } \theta \leftarrow \theta - \alpha \frac{1}{|B|} \sum_{i \in B} \nabla J_i$$

Per-row gradient ($\frac{1}{2}$ -squared loss, $e = \hat{y} - y$): $\nabla J_i = [e_i, e_i x_i]$ for (θ_0, θ_1) .

	BGD	SGD	Mini-batch (size b)
Rows per update	m	1	b
Updates per epoch	1	m	$\lceil m/b \rceil$
Cost per update	$O(md)$	$O(d)$	$O(bd)$
Path	smooth	very noisy	mildly noisy
Hardware	may not fit memory	under-uses GPU	best for GPUs

Cost per **epoch** is the same $O(md)$ for all; SGD just makes m times more (cheaper, noisier) updates. $b = 1 \rightarrow$ SGD; $b = m \rightarrow$ BGD.

Why SGD works – "unbiased but noisy"

SGD randomises **which sample** i_t is used. Averaged over the random choice: $\mathbb{E}[\nabla J_{i_t}] = \frac{1}{m} \sum_i \nabla J_i = \nabla J$. So **on average it points the right way (unbiased)**, but each single step can be off (high variance).

Consequences:

- The loss can go **up** after one step; judge the trend over many steps.
- With a fixed α it keeps bouncing near the minimum; a decaying α lets it settle.
- Noise can help escape shallow local minima / saddle points (non-convex problems).
- **Shuffle every epoch**, otherwise consecutive updates follow the data's ordering pattern instead of the true signal.

Worked: one SGD epoch, data (1,2), (2,4), (3,6), (4,8), $\theta = (0,0)$, $\alpha = 0.1$, order S_2, S_4, S_1, S_3

Rule: $e = \hat{y} - y$, $\theta_0 \leftarrow \theta_0 - \alpha e$, $\theta_1 \leftarrow \theta_1 - \alpha e x$. **Each step uses the θ just updated.**

Step	Sample (x, y)	$\hat{y} = \theta_0 + \theta_1 x$	e	θ_0 new	θ_1 new
1	(2, 4)	0	-4	$0 + 0.4 = 0.4$	$0 + 0.1 \cdot 4 \cdot 2 = 0.8$
2	(4, 8)	$0.4 + 3.2 = 3.6$	-4.4	0.84	$0.8 + 1.76 = 2.56$
3	(1, 2)	$0.84 + 2.56 = 3.4$	+1.4	0.70	$2.56 - 0.14 = 2.42$
4	(3, 6)	$0.7 + 7.26 = 7.96$	+1.96	0.504	$2.42 - 0.588 = 1.832$

4 updates in one epoch (BGD made 1). θ bounced past the answer ($\theta_1 = 2.56$, then fell) — that's SGD noise.

Worked: mini-batch $b = 2$, batches $\{S2, S4\}$ then $\{S1, S3\}$

- ① Batch 1 at $\theta = (0,0)$: $S2$ $e = 0 - 4 = -4$, $S4$ $e = 0 - 8 = -8$. Mean $e = -6 \rightarrow \theta_0 = 0.6$. Mean $e \cdot x = (-4 \cdot 2 + -8 \cdot 4)/2 = -20 \rightarrow \theta_1 = 2.0$. $\theta = (0.6, 2.0)$.
- ② Batch 2 at $\theta = (0.6, 2.0)$: $S1$ $\hat{y} = 2.6$, $e = 0.6$; $S3$ $\hat{y} = 6.6$, $e = 0.6$. Mean $e = 0.6 \rightarrow \theta_0 = 0.54$. Mean $e \cdot x = (0.6 \cdot 1 + 0.6 \cdot 3)/2 = 1.2 \rightarrow \theta_1 = 1.88$. $\theta = (0.54, 1.88)$.
- ③ Updates per epoch = $\lceil 4/2 \rceil = 2$. E.g. $m = 1000$, $b = 64 \rightarrow \lceil 15.6 \rceil = 16$ (the last batch is smaller; average over its real size).

⚠ Exam traps

- Summing the gradients over all samples and updating once after the loop = BGD, not SGD.
- Batch sizes 32/64/128 are chosen for GPU efficiency, not because maths requires them.
- Most stable = BGD, noisiest = SGD; mini-batch = the best overall balance.
- Lab versions use the full squared error: $w \leftarrow w - \alpha \cdot 2(wx_i - y_i)x_i$, seed 42, shuffle the indices each epoch.

7 Evaluation Metrics

Big idea: the training loss helps the optimiser; metrics judge the model on data it hasn't seen. Here the residual is $e = y - \hat{y}$.

➤ Regression metrics

Regression formulas

$$\text{MAE} = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

$$\text{MSE} = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\text{MSE}}$$

$$\text{MAPE} = \frac{100}{n} \sum \frac{|y_i - \hat{y}_i|}{|y_i|}$$

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

$$R_{adj}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1}$$

$\text{RSS} = \sum (y_i - \hat{y}_i)^2$ (model's error), $\text{TSS} = \sum (y_i - \bar{y})^2$ (error of always predicting the mean of the **actual** y). p = number of features (no intercept).

Metric	Unit	Key property
MAE	same as y	typical miss; robust; every error counts linearly
MSE	squared units	punishes big errors (5 vs 2 $\rightarrow$ 25 vs 4 = 6.25x)
RMSE	same as y	keeps MSE's outlier sensitivity; RMSE $\geq$ MAE (a big gap = some large errors)
MAPE	%	relative error; undefined if any $y = 0$; blows up for small y
R^2	none	1 = perfect, 0 = as good as the mean, negative = worse than the mean
Adj. R^2	none	penalises extra features; can fall when a useless feature is added

Worked: $y = 40, 50, 60, 70, 80$ and $\hat{y} = 45, 52, 58, 72, 78$

- 1 Errors $y - \hat{y}$: -5, -2, 2, -2, 2. Absolute: 5, 2, 2, 2, 2 (sum 13). Squared: 25, 4, 4, 4, 4 (sum 41).
- 2 $\text{MAE} = 13/5 = 2.6$; $\text{MSE} = 41/5 = 8.2$; $\text{RMSE} = \sqrt{8.2} = 2.86$.
- 3 APE : $5/40 = 12.5\%$, $2/50 = 4\%$, $2/60 = 3.33\%$, $2/70 = 2.86\%$, $2/80 = 2.5\% \rightarrow \text{MAPE} = 25.19/5 = 5.04\%$.
- 4 $\bar{y} = 60 \rightarrow \text{TSS} = 400 + 100 + 0 + 100 + 400 = 1000$; $\text{RSS} = 41 \rightarrow R^2 = 1 - 41/1000 = 0.959$.
- 5 Adjusted, $n = 5$, $p = 1$: $1 - (1 - 0.959) \cdot \frac{4}{3} = 1 - 0.041 \times 1.333 = 0.945$.

Why adjusted R^2 exists (the logic chain)

Add any feature $\rightarrow$ OLS can fit a little noise $\rightarrow$ RSS falls a little $\rightarrow R^2 = 1 - \text{RSS}/\text{TSS}$ rises regardless of the reason $\rightarrow R^2$ falsely reports improvement $\rightarrow$ a fair metric must penalise complexity $\rightarrow$ adjusted R^2 . Its factor $\frac{n-1}{n-p-1}$ grows with p , so a useless feature lowers it.

e.g. 60 rows: model 1 ($p = 3$, $R^2 = 0.750$) $\rightarrow$ adj 0.7366; model 2 ($p = 5$, $R^2 = 0.758$) $\rightarrow$ adj 0.7356 $\rightarrow$ prefer model 1.

➤ Classification metrics

	Predicted 0	Predicted 1
Actual 0	TN	FP (false alarm)
Actual 1	FN (missed)	TP

sklearn: rows = actual, columns = predicted; confusion_matrix(...).ravel() → tn, fp, fn, tp.

Classification formulas

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (TPR)} = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN}$$

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \text{ FPR} = 1 - \text{spec}$$

- **Precision** = of the ones I **predicted** positive, how many were right → matters when **false alarms are costly** (spam folder hiding real mail).
- **Recall** = of the **actual** positives, how many I caught → matters when **misses are costly** (fraud, cancer).
- F_1 = harmonic mean (punishes imbalance between P and R); ignores TN. $\beta = 2$ favours recall; $\beta = 0.5$ favours precision.
- Accuracy lies on imbalanced data — check per-class recall.

Worked: TP = 40, FP = 10, FN = 20, TN = 30

Accuracy = $70/100 = 0.70$. Precision = $40/50 = 0.80$. Recall = $40/60 = 0.667$. $F_1 = 2(0.8)(0.667)/(1.467) = 0.727$ (check: $80/110$). Specificity = $30/40 = 0.75$.

➤ Multiclass averaging

Averages

$$\text{Macro} = \frac{1}{K} \sum_k M_k \quad \text{Weighted} = \frac{\sum_k n_k M_k}{\sum_k n_k} \quad \text{Micro: pool all TP, FP, FN first}$$

- **Macro**: every class gets an equal vote → use when **all classes are equally important** (exposes a weak small class).
- **Weighted**: big classes count more (by support n_k).
- **Micro**: for single-label multiclass, **micro-P = micro-R = micro-F1 = accuracy**.

e.g. class $F_1 = 0.9, 0.6, 0.3$ with supports 50, 30, 20: macro = $1.8/3 = 0.60$; weighted = $(45 + 18 + 6)/100 = 0.69$.
Model A F_1 s (0.95, 0.90, 0.20) → macro 0.683; model B (0.75, 0.75, 0.75) → macro 0.75 → B is better if all classes matter equally.

➤ ROC and AUC (extra)

Lowering the threshold → more positives → $TPR \uparrow$ and $FPR \uparrow$. ROC = TPR vs FPR over all thresholds. **AUC** = 1 perfect, 0.5 random; **AUC** = $P(\text{a random positive scores higher than a random negative})$.

⚠ Exam traps

- Accuracy = $(TP + TN)/\text{total}$, not $(TP + FP)/\text{total}$.
- MAPE divides by the **actual** y . TSS uses the mean of the **actual** y .
- Adjusted R^2 denominator is $n - p - 1$. sklearn MAPE returns a fraction ($\times 100$ for %).
- Two models with the same MAE can have different MAPE (errors on small actual values cost more %).

8 Polynomial Regression & Assumptions of Linear Regression

➤ Polynomial regression

Big idea: a straight line can't follow a curve, so create new features $x^2, x^3, \dots$ and run ordinary linear regression on them.

Polynomial model

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_n x^n \quad \phi(x) = (1, x, x^2, \dots, x^n)$$

Design matrix columns: $1, x, x^2, \dots \rightarrow$ still $\beta = (X^T X)^{-1} X^T y$. Still linear regression because it's linear in the β 's.
Degree n , 1 feature $\rightarrow n + 1$ columns; turning points $\leq n - 1$. Degree 2 on 2 inputs $\rightarrow 1, x_1, x_2, x_1^2, x_1 x_2, x_2^2 = 6$ columns.

A curve in (x, y) becomes a **plane** in (x, x^2, y) space — the model fits a flat plane to lifted data. Higher degree $\rightarrow$ lower **training** error always, but **too high a degree overfits** (wiggly curve through noise, large test error). Pick the degree on validation data. Scale x before using high powers (x^8 explodes).

Lab rule ("Polynomial Detective"): underfit if $\text{train} > 0.3$ and $\text{test} > 0.3$ and $|\text{train} - \text{test}| < 0.05$; else overfit if $\text{train} < 0.1$ and $\text{test} > 0.15$; else good. Best degree = lowest test MSE (tie $\rightarrow$ smallest degree).

➤ The 5 assumptions (check with residuals $e = y - \hat{y}$)

Assumption	Means	How to see violation	If violated
1. Linearity	y is linear in the features (central trend)	U-shape / curve in residuals vs $\hat{y}$	add x^2 , transform
2. Normality of residuals	$\epsilon \sim \mathcal{N}(0, \sigma^2)$	skewed histogram; Q-Q plot off the line	CI's/p-values unreliable (prediction OK)
3. Homoscedasticity	constant error variance	funnel shape (spread grows with $\hat{y}$)	log-transform y ; standard errors wrong
4. No autocorrelation	errors independent (over time/order)	waves/runs over time; Durbin-Watson far from 2	time-series models
5. No multicollinearity	features not linear combos of each other	correlation $\approx \pm 1$, high VIF	unstable coefficients; drop/combine, Ridge

Funnel = heteroscedasticity; **U-shape** = non-linearity. Normality is about the **residuals**, not the features.

Proof – perfect multicollinearity makes $X^T X$ singular

- ① Suppose column 3 = column 1 + column 2. Then $X\mathbf{c} = \mathbf{0}$ for $\mathbf{c} = (1, 1, -1)^T \neq \mathbf{0}$.
☞ $\text{col1} + \text{col2} - \text{col3} = 0$ in every row
- ② $X^T X \mathbf{c} = X^T (X \mathbf{c}) = X^T \mathbf{0} = \mathbf{0}$
☞ matrix multiplication can be regrouped
- ③ If $(X^T X)^{-1}$ existed: $\mathbf{c} = (X^T X)^{-1} \mathbf{0} = \mathbf{0}$ – contradiction, since $\mathbf{c} \neq \mathbf{0}$.
☞ multiply step 2 by the inverse
- ④ So $X^T X$ has no inverse: $\det(X^T X) = 0$ and OLS has no unique solution.

Example: $X = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 3 \\ 1 & 3 & 4 \end{bmatrix}$ ($\text{col3} = \text{col1} + \text{col2}$). A scaled copy ($x_2 = 2x_1$, $^{\circ}\text{F} = 1.8^{\circ}\text{C} + 32$) works the same way.

Formal tests (extra)

$$\text{VIF}_j = \frac{1}{1 - R_j^2} \quad DW = \frac{\sum_{t=2}^n (e_t - e_{t-1})^2}{\sum_{t=1}^n e_t^2} \in [0, 4]$$

$R_j^2 = R^2$ from regressing feature j on the **other features**. $\text{VIF} > 10$ = serious ($R_j^2 = 0.9 \rightarrow \text{VIF } 10$). $DW \approx 2$ = no autocorrelation; < 2 positive; > 2 negative. e.g. residuals 1, -1, 1, -1: numerator $4 + 4 + 4 = 12$, denominator 4 $\rightarrow DW = 3$.

⚠ Exam traps

- Multicollinearity hurts the **coefficients** (interpretation) more than prediction.
- Autocorrelation is about the residuals over time; multicollinearity is about features vs features.
- Create x^2 from the training x and apply the same transform to the test x .

9 Bias, Variance & the Tradeoff

Big idea: a model's expected test error splits into three parts: Bias^2 (too simple) + Variance (too sensitive to the sample) + σ^2 (noise nobody can remove).

► The setup

Data-generating model

$$Y = f(X) + \varepsilon, \quad E(\varepsilon) = 0, \quad \text{Var}(\varepsilon) = \sigma^2 = E(\varepsilon^2) - [E(\varepsilon)]^2 = E(\varepsilon^2)$$

f = the true (population) function; ε = white noise $\rightarrow$ **irreducible error**. No model can remove it.

- The fitted model $\hat{f}$ depends on **which training sample** you got $\rightarrow \hat{f}(x)$ is a **random variable** (an **estimator**: sample mean, linear regression, decision tree...). Fit on many samples $\rightarrow$ many different $\hat{f}$'s.

Bias, variance, decomposition

$$\text{Bias} = E[\hat{f}(x)] - f(x) \quad \text{Var} = E[(\hat{f}(x) - E[\hat{f}(x)])^2]$$

$$\text{MSE} = E[(Y - \hat{f}(x))^2] = \text{Bias}^2 + \text{Var} + \sigma^2$$

Derivation of the decomposition (write $\hat{f} = \hat{f}(x)$, $\bar{f} = E[\hat{f}]$)

- $E[(Y - \hat{f})^2] = E[(f + \varepsilon - \hat{f})^2]$
 $\hookrightarrow$ replace Y by $f + \varepsilon$
- $= E[(f - \hat{f})^2] + 2E[\varepsilon(f - \hat{f})] + E[\varepsilon^2]$
 $\hookrightarrow$ expand $(A + \varepsilon)^2 = A^2 + 2A\varepsilon + \varepsilon^2$ with $A = f - \hat{f}$
- Middle term $= 2E[\varepsilon] E[f - \hat{f}] = 2 \cdot 0 \cdot (\dots) = 0$
 $\hookrightarrow$ new noise ε is independent of the trained model, and $E(\varepsilon) = 0$
 ; last term $= \sigma^2$.
- $E[(f - \hat{f})^2] = E[(f - \bar{f} + \bar{f} - \hat{f})^2]$
 $\hookrightarrow$ add and subtract the average prediction $\bar{f}$
- $= (f - \bar{f})^2 + 2(f - \bar{f})E[\bar{f} - \hat{f}] + E[(\bar{f} - \hat{f})^2]$
 $\hookrightarrow$ expand; $(f - \bar{f})$ is a fixed number, so it comes out of E
- $E[\bar{f} - \hat{f}] = \bar{f} - E[\hat{f}] = 0 \rightarrow$ middle term vanishes.
 $\hookrightarrow$ definition of $\bar{f}$
- $= \underbrace{(f - \bar{f})^2}_{\text{Bias}^2} + \underbrace{E[(\bar{f} - \hat{f})^2]}_{\text{Var}} \rightarrow \text{total} = \text{Bias}^2 + \text{Var} + \sigma^2$

Worked

(a) $E[\hat{f}(x)] = 12$, $f(x) = 10 \rightarrow \text{Bias} = 12 - 10 = 2$. (b) $\text{Bias}^2 = 1$, $\text{Var} = 9$, $\sigma^2 = 2 \rightarrow \text{MSE} = 1 + 9 + 2 = 12$; **Variance dominates** $\rightarrow$ overfitting. (c) Three models predict 9, 11, 13 at x where $f = 10$: mean $= 11 \rightarrow \text{Bias} = 1$; $\text{Var} = ((-2)^2 + 0^2 + 2^2)/3 = 8/3 = 2.67$.

► Underfit vs overfit

	High bias (underfit)	High variance (overfit)
Model	too simple (line for $y = x^2$)	too complex (degree-15 curve)
Fits on 3 samples	all similar , all wrong	each passes through its points; very different from each other
Train error	high	very low
Validation error	high (close to train)	high (big gap)
Fix	more complex model, more features	more data, regularisation, simpler model, feature selection
Dartboard	tight cluster, off-centre	scattered around the centre

Tradeoff (U-curve): as complexity $\uparrow$, bias² $\downarrow$ and variance $\uparrow$; total error is U-shaped $\rightarrow$ pick the complexity at the bottom of the validation curve (not where bias = 0). σ^2 just lifts the whole curve.

➤ In practice: split, cross-validate, learning curves

- We can't see f , so we estimate generalisation with a **validation set**. A big train-validation gap $\rightarrow$ variance; both high $\rightarrow$ bias.
- **k-fold CV:** split the training data into k folds; train on $k - 1$ folds, validate on the remaining one; repeat k times so every point is validated exactly once. Score = average of the k errors. Each model trains on $n \frac{k-1}{k}$ rows (5-fold on 1000 $\rightarrow$ 800). Fold scores varying a lot $\rightarrow$ high variance.
- **Learning curves** (error vs training size): high bias $\rightarrow$ both curves plateau high and close (more data won't help); high variance $\rightarrow$ big gap that shrinks with more data (more data helps).

➤ Hyperparameter tuning (grid / random search)

Choose hyperparameters by the mean CV score on the training set, refit the winner on all training data, test once. **Grid search** tries every combination: fits = $k \times$ (product of list sizes), e.g. 5 degrees $\times$ 4 λ $\times$ 5 folds = 100 fits. **Random search** tries n_iter sampled combinations: fits = $k \times n_iter$; usually as good with far fewer fits. Search λ on a log scale; put scalers inside a Pipeline (no leakage).

⚠ Exam traps

- $MSE = Bias^2 + Var + \sigma^2$ (bias is squared; variance is not).
- Irreducible error can't be reduced by a more complex model.
- An underfit model is **stable** (low variance) — consistently wrong.

10 Feature Selection

Big idea: keep the original columns that actually help and drop the rest → less overfitting, faster training, easier explanation. (Selection keeps columns; PCA creates new ones.)

➤ Why: the curse of dimensionality

Grid cells

$$\# \text{cells} = b^p \quad (b \text{ bins per feature, } p \text{ features})$$

$b = 10, p = 6 \rightarrow 10^6$ cells. The data can't fill them: most cells are empty, so points are far apart and models overfit. It's b^p , not $p \times b$.

➤ Filter methods (score each feature with a simple rule – no model training)

1. **Duplicate columns** → drop copies.
2. **Variance threshold** → drop near-constant columns. $\text{Var} = \frac{1}{n} \sum (x_i - \bar{x})^2$; for a 0/1 feature with fraction q of ones, $\text{Var} = q(1 - q)$ (10% ones → $0.1 \times 0.9 = 0.09$). **Variance ≠ relevance** (a student ID has huge variance and no meaning); variance depends on units.
3. **Pearson correlation with the target** → keep features with large $|r|$.
4. **Correlation between inputs** → from a pair with $|r| \approx 1$, drop one (redundant).

Pearson correlation

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2} \sqrt{\sum (y_i - \bar{y})^2}} \in [-1, 1]$$

Worked: $r = 0$ although y depends perfectly on x

$x = -2, -1, 0, 1, 2$ and $y = x^2 = 4, 1, 0, 1, 4$. $\bar{x} = 0, \bar{y} = 10/5 = 2$.

- ① $(x - \bar{x})(y - \bar{y})$: $(-2)(2) = -4, (-1)(-1) = 1, (0)(-2) = 0, (1)(-1) = -1, (2)(2) = 4$.
- ② $\text{Sum} = -4 + 1 + 0 - 1 + 4 = 0 \rightarrow r = 0$.

Pearson only sees linear relationships — a correlation filter would wrongly drop x . ($r = 0$ does not mean independent.)

Normal case: $x = 1, 2, 3$ and $y = 2, 4, 7 \rightarrow S_{xy} = 5, S_{xx} = 2, S_{yy} = 12.667 \rightarrow r = 5/\sqrt{2 \times 12.667} = 0.993$.

➤ Wrapper methods (train and evaluate the model on subsets)

generate a candidate subset → train the model → evaluate on validation/CV → generate the next subset

Method	How	Models trained
Exhaustive	every non-empty subset	$2^p - 1$ ($p = 12 \rightarrow 4095$)
Forward selection	start empty; each round add the feature that helps most	$p + (p - 1) + \dots + 1 = \frac{p(p+1)}{2}$
Backward elimination	start with all; each round remove the least useful	$\approx \frac{p(p+1)}{2}$
RFE	fit, drop the feature with the smallest importance, refit...	one fit per feature removed

Why $p(p+1)/2$: round 1 tries p features, round 2 tries the remaining $p - 1$, ..., last round $1 \rightarrow 1 + 2 + \dots + p = p(p+1)/2$ ($p = 10 \rightarrow 55$, vs 1023 exhaustive). Choose the best subset **along the path**, not the last one.

	Filter	Wrapper	Embedded
Uses model?	no	yes, many times	during training (Lasso, tree importance)
Speed	fast	slow	medium
Weakness	misses interactions/non-linear	expensive; can overfit the selection	model-specific

Workflow / leakage

Split first $\rightarrow$ filter on train $\rightarrow$ wrapper with validation/CV $\rightarrow$ test once. Selecting features on the full dataset before CV is leakage.

11 Principal Component Analysis (PCA)

Big idea: rotate the axes so the first new axis (PC1) points along the direction of greatest spread (variance), PC2 the next greatest at right angles, Keep the top $k \rightarrow$ fewer dimensions with most of the information (information = variance). PCA is **unsupervised** (never uses y) and builds **new** features (extraction), unlike selection.

Steps

1. **Centre:** $X_c = X - \mathbf{1}\mu^T$ (subtract each column's training mean $\rightarrow$ every column has mean 0). **Standardise** too if units differ (cm vs $\text{\AA}$), otherwise the big-unit feature dominates PC1.
2. **Covariance matrix:** $S = \frac{1}{n-1} X_c^T X_c$ ($p \times p$, symmetric: $S_{jk} = S_{kj}$; diagonal = variances).
3. **Eigen-decomposition:** solve $S\mathbf{v} = \lambda\mathbf{v}$. Eigenvectors $\mathbf{v} =$ PC directions (unit length, orthogonal); eigenvalues $\lambda =$ variance along each PC.
4. **Sort** λ descending; keep the top k eigenvectors W_k .
5. **Project:** $Z = X_c W_k$ ($n \times k$ scores).

Derivation – variance along a direction is $\mathbf{u}^T S \mathbf{u}$, and the best $\mathbf{u}$ is an eigenvector

1. Projection of a centred point onto a unit vector $\mathbf{u}$: $z_i = \mathbf{u}^T \mathbf{x}_i$
 $\hookrightarrow$ the coordinate of the point along $\mathbf{u}$ ($\mathbf{u}$ must have length 1)
2. Mean of the $z_i = 0$ (data centred), so $\text{Var}(z) = \frac{1}{n-1} \sum z_i^2$
3. $= \frac{1}{n-1} \sum (\mathbf{u}^T \mathbf{x}_i)(\mathbf{x}_i^T \mathbf{u}) = \mathbf{u}^T \left[\frac{1}{n-1} \sum \mathbf{x}_i \mathbf{x}_i^T \right] \mathbf{u} = \mathbf{u}^T S \mathbf{u}$
 $\hookrightarrow z_i^2 = z_i \cdot z_i$ and a number equals its transpose; $\mathbf{u}$ comes out of the sum
4. Maximise $\mathbf{u}^T S \mathbf{u}$ subject to $\mathbf{u}^T \mathbf{u} = 1$ (Lagrange): $\frac{\partial}{\partial \mathbf{u}} [\mathbf{u}^T S \mathbf{u} - \lambda(\mathbf{u}^T \mathbf{u} - 1)] = 2S\mathbf{u} - 2\lambda\mathbf{u} = 0$
 $\hookrightarrow$ identity 3 from Topic 4, twice
5. $\Rightarrow S\mathbf{u} = \lambda\mathbf{u} \rightarrow \mathbf{u}$ must be an **eigenvector**; then $\text{Var} = \mathbf{u}^T S \mathbf{u} = \lambda \mathbf{u}^T \mathbf{u} = \lambda \rightarrow$ the **largest eigenvalue's eigenvector gives the most variance = PC1**.

PCA formulas

$$|S - \lambda I| = 0 \quad (2 \times 2: \lambda^2 - \text{tr}(S)\lambda + \det S = 0) \quad \text{EVR}_j = \frac{\lambda_j}{\sum_{\ell} \lambda_{\ell}} \quad \sum_j \lambda_j = \text{trace}(S)$$

Worked: points (2,1), (3,3), (4,5)

1. Mean = (3, 3). Centred: (-1, -2), (0, 0), (1, 2).
2. $S = \frac{1}{2} \begin{bmatrix} 1+0+1 & 2+0+2 \\ 2+0+2 & 4+0+4 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$
 $\hookrightarrow$ entry $(j,k) = \Sigma(\text{product of centred column } j \text{ and column } k)/(n-1)$
3. trace = 5, det = $1 \cdot 4 - 2 \cdot 2 = 0 \rightarrow \lambda^2 - 5\lambda + 0 = 0 \rightarrow \lambda = 5, 0$.
4. Eigenvector for $\lambda = 5$: $(1-5)v_1 + 2v_2 = 0 \rightarrow v_2 = 2v_1 \rightarrow \mathbf{v} = (1, 2)/\sqrt{5} = (0.447, 0.894)$.
 $\hookrightarrow$ normalise to length 1
5. $\text{EVR}(\text{PC1}) = 5/(5+0) = 100\%$ (the points lie on a line $\rightarrow$ 1-D is lossless).
6. Score of (-1, -2): $z = (-1 \cdot 1 - 2 \cdot 2)/\sqrt{5} = -5/\sqrt{5} = -2.236$.

Another: $S = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \rightarrow \lambda^2 - 6\lambda + 8 = 0 \rightarrow \lambda = 4, 2$; $\text{PC1} = (1,1)/\sqrt{2}$, $\text{EVR} = 4/6 = 66.7\%$.

► Choosing k , and limits

- **Cumulative EVR**: smallest k reaching the target (e.g. $\geq 90\%$). $\lambda = 6, 2, 1, 0.5 \rightarrow 63.2\%, 84.2\%, 94.7\% \rightarrow k = 3$. **Scree plot**: λ vs component; look for the elbow.
- **Reconstruction**: $\hat{X}_c = ZW_k^T$; the top- k PCs give the **smallest reconstruction error** (same as maximum variance).
- **PC scores** are **uncorrelated** ($\text{Cov}(z_r, z_s) = 0$) but **not necessarily independent**.
- **When PCA fails**: a low-variance direction can be the predictive one (it's unsupervised); curved structure (linear PCA can't unroll it); unscaled features; outliers. Fitting PCA before CV = leakage.
- **SVD route** (what sklearn does): $X_c = U\Sigma V^T \rightarrow$ directions = columns of V , $\lambda_j = s_j^2/(n-1)$, scores = $U\Sigma$.

⚠ Exam traps

- `np.linalg.eigh` returns eigenvalues in **ascending** order — reverse before picking the top k .
- Eigenvector sign is arbitrary (v and $-v$ are the same PC).
- Eigenvalue = variance (not std). sklearn PCA centres but does **not** standardise.

12 Regularization (Ridge L2, Lasso L1, Elastic Net)

Big idea: overfitted models have **huge coefficients** (a slope of 4500 swings $\hat{y}$ wildly for a tiny change in x).
Add a penalty on coefficient size to the loss $\rightarrow$ smaller, calmer coefficients $\rightarrow$ a bit more bias, much less variance.

Penalised losses

$$\text{Ridge: } \sum (y_i - \hat{y}_i)^2 + \lambda \sum_{j \geq 1} w_j^2 \quad \text{Lasso: } \sum (y_i - \hat{y}_i)^2 + \lambda \sum_{j \geq 1} |w_j|$$

$\lambda \geq 0$ sets the strength ($\lambda = 0 \rightarrow$ OLS; $\lambda \rightarrow \infty \rightarrow$ all slopes $\rightarrow 0$, a flat line = underfit). The intercept is **not** penalised.

Derivation – Ridge slope in 1-D

- 1 Same trick as OLS: $c = \bar{y} - m\bar{x}$, so with $a_i = x_i - \bar{x}$, $b_i = y_i - \bar{y}$: $E(m) = \sum (b_i - ma_i)^2 + \lambda m^2$
- 2 $\frac{dE}{dm} = -2 \sum a_i b_i + 2m \sum a_i^2 + 2\lambda m$
👉 OLS part from Topic 3, plus the derivative of $\lambda m^2 = 2\lambda m$
- 3 Set to 0: $m (\sum a_i^2 + \lambda) = \sum a_i b_i$
- 4
$$m_{\text{ridge}} = \frac{S_{xy}}{S_{xx} + \lambda}$$

👉 λ sits in the **denominator** $\rightarrow$ m shrinks toward 0 but never reaches exactly 0

Derivation – Ridge normal equation (many features)

- 1 $L = (\mathbf{y} - \mathbf{X}\mathbf{w})^T (\mathbf{y} - \mathbf{X}\mathbf{w}) + \lambda \mathbf{w}^T \mathbf{w}$
- 2 $\nabla L = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \mathbf{w} + 2\lambda \mathbf{w}$
👉 OLS gradient (Topic 4) + derivative of $\lambda \mathbf{w}^T \mathbf{w} = 2\lambda \mathbf{w}$ (identity 3 with $A = I$)
- 3 Set to 0: $(\mathbf{X}^T \mathbf{X} + \lambda I) \mathbf{w} = \mathbf{X}^T \mathbf{y}$
👉 $\lambda \mathbf{w} = \lambda I \mathbf{w}$, then factor out $\mathbf{w}$
- 4
$$\mathbf{w}_{\text{ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda I)^{-1} \mathbf{X}^T \mathbf{y}$$

👉 adding λI also makes the matrix invertible even with collinear features

Derivation – Lasso slope in 1-D (why exact zeros)

- 1 $E(m) = \sum (b_i - ma_i)^2 + \lambda |m|$. $|m|$ has a kink at 0, so handle $m > 0$ and $m < 0$ separately.
- 2 Case $m > 0$ ($|m| = m$): $\frac{dE}{dm} = -2S_{xy} + 2mS_{xx} + \lambda = 0 \rightarrow m = \frac{S_{xy} - \lambda/2}{S_{xx}}$.
👉 the worksheet absorbs the $1/2$ into λ and writes $m = \frac{S_{xy} - \lambda}{S_{xx}}$
- 3 Case $m < 0$ ($|m| = -m$): $m = \frac{S_{xy} + \lambda}{S_{xx}}$ (worksheet form).
- 4 λ is subtracted in the numerator $\rightarrow$ once $\lambda \geq |S_{xy}|$ the formula would cross 0 and break its own sign assumption $\rightarrow$ the algorithm **stops at exactly $m = 0$** . That's sparsity = automatic feature selection.

Worked: $S_{xy} = 100$, $S_{xx} = 50$ (OLS $m = 2$)

λ	Ridge $m = 100/(50 + \lambda)$	Lasso $m = (100 - \lambda)/50$
0	2	2
50	1	1
100	0.667	0
150	0.5 (never 0)	formula $-1 \rightarrow$ clipped to 0

$m < 0$ case: $S_{xy} = -100 \rightarrow \lambda = 100$ gives 0; $\lambda = 150$ gives $+1$, which breaks $m < 0 \rightarrow$ also clipped to 0.

➤ Ridge vs Lasso vs Elastic Net

	Ridge (L2)	Lasso (L1)	Elastic Net
Penalty	$\lambda \sum w_j^2$	$\lambda \sum w_j $	$\lambda_1 \sum w_j + \lambda_2 \sum w_j^2$
Effect	shrinks all, never exactly 0	exact zeros (feature selection)	zeros + groups correlated features
Closed form	yes	no (coordinate descent)	no
Shape (2-D)	circle $w_1^2 + w_2^2 \leq r^2$	diamond $ w_1 + w_2 \leq r$ (corners on axes)	between the two
Best when	all features useful; multicollinearity	many useless features; interpretability	many features + correlated groups

- **Geometry:** the OLS error ellipses expand until they first touch the constraint region. A circle has no corners $\rightarrow$ the touch point is rarely on an axis. A diamond has **corners on the axes** $\rightarrow$ the touch point is often a corner $\rightarrow$ **a coefficient is exactly 0**.
- Ridge shrinks big coefficients fastest ($w = 5000$ adds $25,000,000\lambda$ to the loss; $w = 1$ adds only λ).
- With correlated features, Lasso keeps one arbitrarily; Ridge/Elastic Net share the weight.
- Elastic Net in sklearn: `ElasticNet(alpha, l1_ratio)`; `l1_ratio = 1` $\rightarrow$ Lasso, `0` $\rightarrow$ Ridge.

Practice rules

Standardise features first (the penalty depends on scale). Don't penalise the intercept. Choose λ by cross-validation on a log grid (0.001, 0.01, 0.1, 1, 10, 100): train error rises with λ , validation error falls then rises — take the bottom. Logistic regression: the L2 penalty adds $2\lambda\beta_j$ to the gradient; sklearn's $C = 1/\lambda$.

⚠ Exam traps

- λ in the **numerator** (Lasso, subtracts $\rightarrow$ zeros) vs the **denominator** (Ridge, divides $\rightarrow$ shrink).
- Lasso formula going past 0 $\rightarrow$ answer 0, not the negative number.
- Ridge adds λI to $X^T X$ (not to $X^T y$). Ridge increases bias slightly and reduces variance.
- Overfitting = **high variance, low bias**.

13 Time Series Analysis

Big idea: ordinary ML assumes rows are **I.I.D.** (shuffle them and nothing changes). In a time series **order is the information**: today depends on yesterday.

- **No shuffled K-fold:** a random split could train on 2026 to predict 2024 → leakage. Use a forward-chaining split (`TimeSeriesSplit`): train on $1..t$, test on $t+1..t+h$, growing the window.
- Classical models assume **equally spaced** time steps.

➤ Components

Additive decomposition

$$y_t = T_t + S_t + R_t$$

Trend T = long-term direction. **Seasonality** S = repeating cycles at a **fixed, known** period (weekend dips, summer AC sales). **Noise / residual** R = random, unpredictable. If a model's residuals look like pure noise, it has captured everything learnable.

➤ Stationarity

Weak stationarity – all three must hold

$$E[y_t] = \mu \text{ (constant mean)} \quad \text{Var}(y_t) = \sigma^2 \text{ (constant variance)} \quad \text{Cov}(y_t, y_{t-k}) = \gamma_k \text{ (depends only on lag } k\text{)}$$

AR/MA models need stationary data. A trend breaks condition 1; growing spread breaks condition 2. (ADF test: null = non-stationary; $p < 0.05 \rightarrow$ stationary.)

➤ Lags and differencing

Lag and differences

$$\text{lag } k = y_{t-k} \quad \Delta y_t = y_t - y_{t-1} \quad \Delta_s y_t = y_t - y_{t-s} \text{ (seasonal)} \quad \hat{y}_{t+1} = y_t + \widehat{\Delta y}_{t+1}$$

Worked: series 10, 13, 15, 20

- 1 First differences: $13 - 10 = 3, 15 - 13 = 2, 20 - 15 = 5 \rightarrow [-, 3, 2, 5]$ (the first value is lost).
- 2 Second difference (difference of the differences): $2 - 3 = -1, 5 - 2 = 3$.
- 3 Lag-1 column = $[-, 10, 13, 15]$.

First differencing removes a (linear) trend; seasonal differencing (lag s) removes seasonality; log stabilises growing variance. "Last week" = lag 1 for weekly data, lag 7 for daily data.

➤ AR(p) – Auto-Regression

AR(p)

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t \quad \text{AR(1) long-run mean } \mu = \frac{c}{1 - \phi_1}$$

Regress the series on its own past p values. p too large ($p = 365$ for tomorrow's temperature) $\rightarrow$ 365 weights $\rightarrow$ slow, overfits noise.

Worked

AR(1): $y_t = 2 + 0.6y_{t-1}$, $y_{t-1} = 10 \rightarrow$ forecast $2 + 6 = 8$ (forecast sets $\varepsilon = 0$); long-run mean $= 2/(1 - 0.6) = 5$. AR(2): $5 + 0.5(20) - 0.2(10) = 5 + 10 - 2 = 13$. Multi-step: earlier forecasts become the inputs.

➤ ACF and PACF

Sample ACF at lag k

$$\rho_k = \frac{\sum_{t=k+1}^N (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^N (y_t - \bar{y})^2} \in [-1, 1] \quad \text{significance band } \pm \frac{1.96}{\sqrt{N}}$$

Worked: $y = 1, 2, 3, 4, 5$, lag 1

- 1 $\bar{y} = 3$; deviations: $-2, -1, 0, 1, 2$. Denominator $= 4 + 1 + 0 + 1 + 4 = 10$.
- 2 Pair each value with the one before it: $(-1)(-2) + (0)(-1) + (1)(0) + (2)(1) = 2 + 0 + 0 + 2 = 4$.
- 3 $\rho_1 = 4/10 = 0.4$.

- **ACF** = total correlation with lag k (direct + "ripple" through the lags in between). $\rho \approx +1$: today follows the past; $\rho \approx -1$: mean-reverting.
- **PACF** = **direct** correlation with lag k after removing the effect of the lags in between. (Day-before-yesterday $\rightarrow$ yesterday $\rightarrow$ today is a ripple; PACF removes it.)

➤ MA(q) – Moving Average of shocks

MA(q)

$$y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q} \quad \varepsilon_t = y_t - \hat{y}_t \text{ (shock / forecast error)}$$

Uses past **errors (shocks)**, not past values — not a rolling mean. e.g. $50 + \varepsilon_t + 0.5\varepsilon_{t-1}$ with $\varepsilon_{-t} = 2$, $\varepsilon_{-t-1} = -4 \rightarrow 50 + 2 - 2 = 50$.

Model	ACF	PACF
AR(p)	tails off slowly	cuts off after lag p
MA(q)	cuts off after lag q	tails off

Memory aid: PACF $\rightarrow p$ (AR), ACF $\rightarrow q$ (MA). ARIMA(p, d, q): d = number of differences. A spike at lag 7 in daily data = weekly seasonality.

⚠ Exam traps

- First differencing removes trend, not seasonality.
- Stationarity $\neq$ constant values; it's constant statistical properties.
- Noise (a component) vs a random shock (one specific unpredictable event).

14 Maximum Likelihood Estimation & Logistic Regression

► Probability vs likelihood

Same formula, different question

Probability: parameter fixed ($p = 0.5$), ask about data ("chance of 7 heads in 10?"). **Likelihood:** data fixed (we saw 7 heads), ask about the parameter ("which p explains this best?"). **MLE:** choose the parameter that makes the observed data most likely: $\hat{\theta} = \arg \max_{\theta} L(\theta; D)$.

Binomial likelihood

$$L(p) = \binom{n}{k} p^k (1-p)^{n-k}$$

Worked: 7 heads in 10 tosses

$\binom{10}{7} = 120$. $L(0.5) = 120 \times 0.5^0 = 0.117$. $L(0.7) = 120 \times 0.7^7 \times 0.3^3 = 120 \times 0.0824 \times 0.027 = \mathbf{0.267}$ (largest). $L(0.9) = 120 \times 0.478 \times 0.001 = 0.057$. $\rightarrow p = 0.7$ explains the data best.

Derivation – the Bernoulli MLE is $\hat{p} = k/n$

- 1 $L(p) \propto p^k (1-p)^{n-k}$
the constant $\binom{n}{k}$ doesn't change where the maximum is
- 2 $\ell(p) = \ln L = k \ln p + (n-k) \ln(1-p)$
take logs (the max is in the same place; products become sums): $\ln a^k = k \ln a$
- 3 $\frac{d\ell}{dp} = \frac{k}{p} - \frac{n-k}{1-p}$
 $\frac{d}{dp} \ln(1-p) = \frac{-1}{1-p}$ (chain rule)
- 4 $\frac{k}{p} = \frac{n-k}{1-p} \rightarrow k(1-p) = (n-k)p$
set to 0, cross-multiply
- 5 $k - kp = np - kp \rightarrow k = np \rightarrow \boxed{\hat{p} = k/n}$
the $-kp$ cancels on both sides
(7/10 = 0.7 ✓)

► Logistic regression: score $\rightarrow$ probability

- Accuracy is a step function (no gradient) and doesn't reward confidence. We need probabilities.
- Linear score** $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ (any real number; not a probability). **Decision boundary:** $z = 0$ (a line/plane).
- A step function ($z \geq 0 \rightarrow 1$) has zero gradient almost everywhere $\rightarrow$ can't learn. Use the smooth **sigmoid**.

Sigmoid

$$\hat{p} = P(y = 1 | \mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}} \quad P(y = 0) = 1 - \hat{p} \quad z = \ln \frac{\hat{p}}{1 - \hat{p}} \text{ (logit)}$$

z	-2	-1	0	1	2	large +	large -
$\sigma(z)$	0.119	0.269	0.5	0.731	0.881	$\rightarrow 1$	$\rightarrow 0$

$\sigma(-z) = 1 - \sigma(z)$. Threshold 0.5 $\Rightarrow z = 0$. e.g. $z = -1 + 0.5(2) + 0.25(4) = 1 \rightarrow \hat{p} = 0.731 \rightarrow \text{class 1}$.

► From likelihood to Binary Cross-Entropy (BCE)

Derivation – BCE is the negative log-likelihood

① One row: $P(\text{observed label}) = \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1 - y_i}$

☞ if $y = 1 \rightarrow \hat{p}$ (the second factor becomes $1 - \hat{p}$); if $y = 0 \rightarrow 1 - \hat{p}$

② All rows (independent): $L(\beta) = \prod_{i=1}^n \hat{p}_i^{y_i} (1 - \hat{p}_i)^{1 - y_i}$

☞ multiply the probabilities

③ $\ell = \ln L = \sum_i [y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)]$

☞ log of a product = sum of logs

④ Maximise $\ell \Leftrightarrow$ minimise $-\ell$. Average over n : $\text{BCE} = -\frac{1}{n} \sum_i [y_i \ln \hat{p}_i + (1 - y_i) \ln(1 - \hat{p}_i)]$

Per row: $y = 1 \rightarrow \text{loss} = -\ln \hat{p}$; $y = 0 \rightarrow \text{loss} = -\ln(1 - \hat{p})$.

Worked: how BCE behaves ($y = 1$)

$\hat{p} = 0.9 \rightarrow -\ln 0.9 = 0.105$ (good). $\hat{p} = 0.5 \rightarrow 0.693$. $\hat{p} = 0.1 \rightarrow -\ln 0.1 = 2.303$ (confidently wrong is very expensive). Two rows $y = [1, 0]$, $\hat{p} = [0.8, 0.3]$: $(-\ln 0.8 - \ln 0.7)/2 = (0.223 + 0.357)/2 = 0.290$.

- BCE is convex but has **no closed form** $\rightarrow$ train with gradient descent (Topic 15).
- Regularised: $J = \text{BCE} + \lambda \cdot R(\beta)$. Large $\lambda \rightarrow$ underfitting.
- Equal accuracy does not mean equal likelihood (confidence matters).

⚠ Exam traps

- For a $y = 0$ row use $1 - \hat{p}$ (not $\hat{p}$).
- z is a score, not a probability. Use the natural log.
- Log-likelihood -20 beats -35 (larger is better).

15 GD on Logistic Regression & Multiclass

Big idea: the BCE gradient simplifies beautifully to $(\hat{p} - y) \cdot x$ — "error × input", the same shape as linear regression.

chain: $\beta_j \rightarrow z = \beta^T \mathbf{x} \rightarrow \hat{p} = \sigma(z) \rightarrow L$ so $\frac{\partial L}{\partial \beta_j} = \frac{\partial L}{\partial \hat{p}} \cdot \frac{\partial \hat{p}}{\partial z} \cdot \frac{\partial z}{\partial \beta_j}$

Link 1 — $\partial L / \partial \hat{p}$

- 1 $L = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})]$
- 2 $\frac{\partial L}{\partial \hat{p}} = -\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}}$
 $\hookrightarrow \frac{d}{d\hat{p}} \ln \hat{p} = \frac{1}{\hat{p}}, \frac{d}{d\hat{p}} \ln(1 - \hat{p}) = -\frac{1}{1-\hat{p}}$, and the two minus signs make it +
- 3 $= \frac{-y(1-\hat{p}) + (1-y)\hat{p}}{\hat{p}(1-\hat{p})}$
 $\hookrightarrow$ common denominator
- 4 $= \frac{-y + y\hat{p} + \hat{p} - y\hat{p}}{\hat{p}(1-\hat{p})} = \frac{\hat{p} - y}{\hat{p}(1-\hat{p})}$
 $\hookrightarrow$ expand; $+y\hat{p}$ and $-y\hat{p}$ cancel

Link 2 — sigmoid derivative $\sigma' = \sigma(1 - \sigma)$

- 1 $\sigma(z) = (1 + e^{-z})^{-1}$
- 2 $\sigma'(z) = -1 \cdot (1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$
 $\hookrightarrow$ power rule, then chain rule: derivative of e^{-z} is $-e^{-z}$
- 3 $= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}$
 $\hookrightarrow$ split into two fractions
- 4 $\frac{e^{-z}}{1 + e^{-z}} = \frac{(1 + e^{-z}) - 1}{1 + e^{-z}} = 1 - \sigma(z)$
 $\hookrightarrow$ add and subtract 1 on top
- 5 $\sigma'(z) = \sigma(z)(1 - \sigma(z)) = \hat{p}(1 - \hat{p})$
 $\hookrightarrow$ largest at $z = 0$: $0.5 \times 0.5 = 0.25$

Link 3 and the result

- 1 $\frac{\partial z}{\partial \beta_j} = x_j$
 $\hookrightarrow z = \beta_0 + \beta_1 x_1 + \dots$; only the β_j term contains β_j ($x_0 = 1$)
- 2 $\frac{\partial L}{\partial \beta_j} = \frac{\hat{p} - y}{\hat{p}(1-\hat{p})} \cdot \hat{p}(1 - \hat{p}) \cdot x_j$
 $\hookrightarrow$ multiply the three links
- 3 $\frac{\partial L}{\partial \beta_j} = (\hat{p} - y) x_j$
 $\hookrightarrow$ the $\hat{p}(1 - \hat{p})$ cancels — so $\frac{\partial L}{\partial z} = \hat{p} - y$
- 4 All n rows: $\nabla J = \frac{1}{n} X^T (\hat{\mathbf{p}} - \mathbf{y})$, $\beta \leftarrow \beta - \alpha \nabla J$
 $\hookrightarrow$ average over rows; $1/n$, no factor 2 (unlike MSE)

Worked: one batch update (worksheet)

Rows ($x = 1, y = 0$) and ($x = 2, y = 1$); $\beta_0 = \beta_1 = 0$; $\alpha = 0.4$.

- ① $z = 0$ for both $\rightarrow \hat{p} = \sigma(0) = 0.5, 0.5$. Errors $\hat{p} - y = +0.5, -0.5$.
- ② $\partial J / \partial \beta_0 = \frac{(0.5)(1) + (-0.5)(1)}{2} = 0$; $\partial J / \partial \beta_1 = \frac{(0.5)(1) + (-0.5)(2)}{2} = \frac{-0.5}{2} = -0.25$.
- ③ $\beta_0 = 0 - 0.4(0) = 0$; $\beta_1 = 0 - 0.4(-0.25) = \mathbf{0.10}$ (update both simultaneously).
- ④ Check: new $z = 0.1, 0.2 \rightarrow \hat{p} = 0.525, 0.550$. BCE: $-\ln(1 - 0.525) = 0.744, -\ln 0.550 = 0.598 \rightarrow$ mean $\mathbf{0.671} < 0.693 = \ln 2$ (the starting loss with $\beta = 0$). Loss fell $\checkmark$.

➤ Multiclass: One-vs-Rest (OvR)

- K classes $\rightarrow$ train K binary logistic models, each "class k vs everything else". Parameters = $K(d + 1)$.
- Predict: compute all K sigmoid scores, pick the **largest**. The K scores need not sum to 1.

➤ Multiclass: Softmax (multinomial logistic regression)

Softmax and cross-entropy

$$\hat{p}_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}} \quad L = - \sum_k y_k \ln \hat{p}_k = - \ln \hat{p}_{\text{true class}} \quad \frac{\partial L}{\partial z_k} = \hat{p}_k - y_k$$

Worked: logits (2, 1, 0)

- ① $e^2 = 7.389, e^1 = 2.718, e^0 = 1 \rightarrow \text{sum} = 11.107$.
- ② $\hat{p} = 7.389/11.107 = \mathbf{0.665}, 2.718/11.107 = \mathbf{0.245}, 1/11.107 = \mathbf{0.090}$ (sum = 1 $\checkmark$).
- ③ True class = 2nd $\rightarrow CE = -\ln 0.245 = \mathbf{1.41}$; if true = 1st $\rightarrow -\ln 0.665 = 0.41$.

Adding the same constant to all logits changes nothing $\rightarrow$ compute with $e^{z_k - \max z}$ to avoid overflow.
Multiplying the logits does change the result. $K = 2$ softmax = sigmoid of $z_1 - z_2$.

	OvR	Softmax
Models	K separate binary models	one joint model
Outputs sum to 1?	no	yes
Loss	K binary BCEs	multiclass cross-entropy

➤ Odds, log-odds, boundaries, regularisation

Odds

$$\text{odds} = \frac{p}{1-p} \quad \ln \frac{p}{1-p} = z = \beta_0 + \beta^T \mathbf{x} \quad \text{odds ratio} = e^{\beta_j}$$

$p = 0.8 \rightarrow \text{odds } 4 \rightarrow \text{log-odds } \ln 4 = 1.386$. One unit more of x_j adds β_j to the **log-odds** (multiplies the odds by e^{β_j}) — not to the probability.

- Assumptions: independent rows, log-odds linear in the features, little multicollinearity.
- Linear $z \rightarrow$ straight boundary. Add $x_1^2, x_2^2 \rightarrow$ **curved** boundary (a circle/ellipse) — still logistic regression.
- Regularisation: $J = \text{BCE} + \lambda \sum \beta_j^2$ (L2 adds $2\lambda \beta_j$ to the gradient $\rightarrow$ shrinks big weights) or $\lambda \sum |\beta_j|$ (L1 $\rightarrow$ zeros). Scale features; don't penalise β_0 . sklearn $C = 1/\lambda$.

⚠ Exam traps

- The gradient is $(\hat{p} - y)x$ when you **subtract** $\alpha \times$ gradient. With $(y - \hat{p})$ you'd go uphill.
- The starting BCE with $\beta = 0$ is always $\ln 2 = 0.693$.
- OvR scores don't sum to 1; softmax probabilities do.



Formulas at a glance

Topic	Must-know formulas
Preprocess	$\frac{x-\min}{\max-\min} \cdot \frac{x-\mu}{\sigma} (\sigma \div n) \cdot \frac{x}{\max x } \cdot \frac{x-\text{med}}{\text{IQR}} \cdot \text{fences } Q_1 - 1.5\text{IQR}, Q_3 + 1.5\text{IQR} \cdot \text{binary cols } \lceil \log_2 N \rceil$
SLR	$m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2} \cdot c = \bar{y} - m\bar{x} \cdot \text{line passes through } (\bar{x}, \bar{y}) \cdot \sum e_i = 0$
MLR	$L = \mathbf{e}^T \mathbf{e} \cdot \nabla L = -2\mathbf{X}^T \mathbf{y} + 2\mathbf{X}^T \mathbf{X} \boldsymbol{\beta} \cdot \mathbf{X}^T \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^T \mathbf{y} \cdot \boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \cdot \text{cost } O(p^3)$
BGD	$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla L \cdot \nabla L = \frac{2}{m} \mathbf{X}^T (\mathbf{X} \boldsymbol{\theta} - \mathbf{y}) \cdot 1 \text{ update/epoch} \cdot \text{diverges if } \alpha \text{ too big}$
SGD / MB	per-row grad $[e, ex], e = \hat{y} - y \cdot \text{updates/epoch: } l, m, [m/b] \cdot \text{cost/update } O(md), O(d) \cdot \text{unbiased but noisy}$
Metrics	$\text{MAE, MSE, RMSE} \geq \text{MAE, MAPE } (\div \text{actual}) \cdot R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} \cdot R_{adj}^2 = 1 - (1 - R^2) \frac{n-1}{n-p-1} \cdot P = \text{TP}/(\text{TP}+\text{FP}) \cdot R = \text{TP}/(\text{TP}+\text{FN}) \cdot \text{FI} = 2\text{PR}/(\text{P}+\text{R}) \cdot \text{Acc} = (\text{TP}+\text{TN})/\text{all} \cdot \text{micro-FI} = \text{accuracy}$
Poly / assump.	degree $n \rightarrow n+1$ cols $\cdot \text{funnel} = \text{heteroscedastic}, U = \text{non-linear} \cdot \mathbf{Xc} = 0 \Rightarrow \det \mathbf{X}^T \mathbf{X} = 0 \cdot \text{VIF} = \frac{1}{1-R_j^2} \cdot D\omega \approx 2$
Bias-var	$\text{Bias} = E[\hat{f}] - f \cdot \text{MSE} = \text{Bias}^2 + \text{Var} + \sigma^2 \cdot k\text{-fold trains on } n(k-1)/k \cdot \text{grid fits} = k \times \Pi(\text{list sizes})$
Selection	$\text{cells} = b^p \cdot \text{Var(binary)} = q(1-q) \cdot \text{Pearson } r \cdot \text{exhaustive } 2^p - 1 \cdot \text{forward/backward } p(p+1)/2$
PCA	$S = \frac{1}{n-1} \mathbf{X}_c^T \mathbf{X}_c \cdot S\mathbf{v} = \lambda \mathbf{v} \cdot \lambda^2 - \text{tr } \lambda + \det = 0 \cdot \text{Var}(u^T \mathbf{x}) = u^T S u \cdot \text{EVR} = \lambda_i / \sum \lambda \cdot Z = \mathbf{X}_c \omega \cdot \lambda = s^2/(n-1)$
Regular.	$\text{Ridge } m = \frac{S_{xy}}{S_{xx} + \lambda}, (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \cdot \text{Lasso } m = \frac{S_{xy} - \lambda}{S_{xx}} \rightarrow \text{clip at } 0 \cdot \text{EN} = L1 + L2$
Time series	$y_t = T + S + R \cdot \text{stationary: const } \mu, \sigma^2, \gamma_k \cdot \Delta y = y_t - y_{t-1} \cdot \text{AR}(p) \cdot \text{MA}(q) \cdot \rho_k \text{ formula} \cdot \text{PACF} \rightarrow p, \text{ACF} \rightarrow q$
MLE / LR	$\hat{p} = k/n \cdot \sigma(z) = \frac{1}{1+e^{-z}} \cdot \text{boundary } z = 0 \cdot \text{BCE} = -\frac{1}{n} \sum [y \ln \hat{p} + (1-y) \ln(1-\hat{p})]$
GD-LR / multi	$\sigma' = \sigma(1-\sigma) \cdot \partial L / \partial \beta_j = (\hat{p} - y) x_j \cdot \nabla J = \mathbf{X}^T (\hat{\mathbf{p}} - \mathbf{y}) / n \cdot \text{OvR } K \text{ models, } K(d+1) \text{ params} \cdot \text{softmax } e^{z_k} / \sum e^{z_j} \cdot \text{CE} = -\ln \hat{p}_{\text{true}} \cdot \text{odds} = p/(1-p)$

Numbers worth remembering

$\sigma(0) = 0.5 \cdot \sigma(1) = 0.731 \cdot \sigma(2) = 0.881 \cdot \ln 2 = 0.693 \cdot \ln 4 = 1.386 \cdot -\ln 0.1 = 2.303 \cdot -\ln 0.9 = 0.105 \cdot e = 2.718 \cdot e^2 = 7.389 \cdot e^{-1} = 0.368 \cdot \text{softmax}(2,1,0) = (0.665, 0.245, 0.090) \cdot \sqrt{2} = 1.414$

All the best for the mid-sem! 🌸